# 블록체인(01) — 시험 직전 정리

중간고사 대면 필기 · 0. Introduction ~ 9. 솔리디티(8주차)

> **시험 직전 체크리스트**
>
> 1. **계산 4종**: UTXO 거스름돈·수수료, 가스 수수료(Legacy / EIP-1559), nBits 목표값, 반감기 보조금
> 2. **비교표**: 합의 알고리즘 4종, 하드/소프트포크, 비트코인 vs 이더리움, EOA vs CA, Optimistic vs ZK
> 3. **순서**: 화폐 연표, 거래 처리 과정, PBFT 5단계, 비트코인 주소 생성
> 4. **솔리디티**: pragma 버전, 저장 영역·복사/참조, msg.sender vs tx.origin, 정수 나눗셈, enum 기본값

## 꼭 외울 숫자

| | | | |
|---|---|---|---|
| **2,100만**<br>BTC 총 발행량 | **210,000블록**<br>반감기 (약 4년) | **2016블록**<br>난이도 재조정 (약 2주) | **10분 / 12초**<br>블록 시간 BTC / ETH |
| **80바이트**<br>BTC 블록 헤더 | **3f+1 / 2f+1**<br>PBFT 노드 / 합의 메시지 | **32 ETH**<br>검증자 예치 | **32슬롯 ≈ 6.4분**<br>에포크 |
| **20바이트**<br>ETH 주소 (Keccak-256 끝) | **$10^{18}$ / $10^{-9}$**<br>1 ETH = $10^{18}$ Wei, 1 gwei = $10^{-9}$ ETH | **6 컨펌**<br>BTC 거래 확정 | **2022.09.15**<br>The Merge (PoW → PoS) |

## 1. 탄생 · 기초

- **연표**: 뉴딜(1933, 금본위제 폐지) → **브레튼우즈(1944)** → **닉슨 쇼크(1971)** → **서브프라임(2008)** → 비트코인 백서(2008) → 제네시스 블록(2009.01.03, "Chancellor on brink of second bailout for banks")
- 전자화폐(카카오페이) / 가상화폐(게임머니) / **암호화폐**(분산원장 + 암호기술, 중앙기관 X)
- eCash(David Chaum, 1983 은닉 서명, **중앙기관이 이중지불 검증**) / B-Money(Wei Dai, 1998, 구현 X)
- **이중 지불**: 같은 화폐를 두 번 사용. 전파 지연·손실로 장부 불일치 → **합의**로 대표 장부 동기화
- **거래 흐름**: 트랜잭션 작성 → 전파 → 각 노드 블록 생성 → 블록 전파 → 합의로 대표 블록 선정 → 기록
- 허가형(승인 노드만, 성능·규제 친화) vs 비허가형(누구나, 탈중앙·낮은 성능) / Public · Private(Hyperledger Fabric, R3 Corda) · Hybrid · Consortium

## 2. 암호기술

| 해시 성질 | 찾기 어려운 것 | 지문 비유 |
|---|---|---|
| 역상 저항성 | y → h(x) = y인 x | 지문만 보고 누구인지 |
| 제2역상 저항성 | x가 주어질 때 같은 해시의 x′ | Alice와 같은 지문의 사람 |
| 충돌 저항성 | 같은 해시인 **아무 쌍** | 지문이 같은 아무 두 사람 |

- 눈사태 효과 / SHA-3 = **Keccak(스펀지 구조)** / Poseidon = ZK 친화
- **머클 트리**: 루트로 전체 무결성. 증명 크기 **O(log N)** (8개 → 형제 해시 3개). Tx3 증명 = {해시4, 해시12, 루트}
- **전자서명**: 개인키로 서명, 공개키로 검증, 해시에 서명 → **무결성 · 출원 인증 · 부인 방지**. RSA는 크고 느림 → **ECDSA(secp256k1)**, EdDSA(Ed25519)
- **집계 서명**(여러 서명 → 하나, BLS, 이더리움) vs **다중 서명**(t-of-N, 2-of-3은 키 1개 유출로 위조 X, 1개 분실해도 사용 O)
- **영지식 증명**: 완전성 · 건전성 · 영지식성 (동굴 예시) → DID, ZK 롤업

## 3. 합의 알고리즘

- 핵심 요소: **안전성 · 생존성 · 최종성 · 장애 허용** / 위협: 지연·손실, 비잔틴 장군, **Sybil**(가짜 노드 대량)
- 분기 선택: 비트코인 **최장 체인(누적 작업량)** → **확률적 최종성** / 이더리움 LMD-GHOST / BFT 계열 2/3 투표 → **즉시 최종성**
- **하드포크**: 비호환, 체인 분리 (BTC↔BCH, ETH↔ETC) / **소프트포크**: 하위 호환, 체인 유지 (**SegWit** 2017)

| | PoW | PoS | DPoS | PBFT |
|---|---|---|---|---|
| 자격 | 해시 연산 | 예치 지분 | 투표로 선출된 대표 | 허가된 참여자 |
| 장점 | 단순, 오랜 운영 | 에너지↓ | 높은 확장성 | **즉시 최종성** |
| 단점 | 자원 소모, **51% 공격** | 부의 집중 | 대표 집중·담합 | 참여자↑ → 통신 폭증 |
| 벌칙 | — | **슬래싱** | 투표로 교체 | — |

- PoW: H(Data ‖ Nonce) < 목표값 (찾기 어렵고 검증 쉬움) / **PBFT**: Request → Pre-Prepare → Prepare → Commit → Reply. 노드 n이면 견디는 f = ⌊(n−1)/3⌋
- **51% 공격**: 상점 B에 보낸 거래로 상품 받음 → 같은 UTXO를 C에 보낸 블록으로 **고의 분기** → 과반 해시파워로 더 긴 체인 공개 → **Longest Chain 규칙**으로 B 거래 취소

## 4. 비트코인

- **주소**: 개인키(256) → 공개키(512, 타원곡선 상수곱) → **SHA-256** → **RIPEMD-160**(160비트) → **Base58**(0, O, I, l 제외)
- 노드: 풀 노드 / **SPV(헤더만)** / Solo Miner / Reference Client(Bitcoin Core). 포트 8333
- **헤더 80바이트**: 버전 4 · 이전 해시 32 · 머클루트 32 · 타임스탬프 4 · Bits 4 · 넌스 4. Tx 홀수면 마지막 복사
- **UTXO**: 잔액 = 미사용 출력 합. 입력 UTXO는 **통째로 소비**, 출력 = 수신자 + 거스름돈. **수수료 = 입력 합 − 출력 합**
- 잠금 **ScriptPubKey** / 해제 **ScriptSig** / P2PKH(표준), P2SH(다중서명)
- **코인베이스**: 블록의 첫 트랜잭션, 이전 UTXO 참조 X, ScriptSig에 임의 데이터, 출력 = 보조금 + 수수료
- **nBits → T = C × 2^(8(E−3))**: E = 상위 1바이트, C = 하위 3바이트. 0x1d00ffff → E = 29, C 뒤 0이 26바이트, 앞 0이 3바이트. 성공 조건 **h < T**
- 보조금: 50 → 25 → 12.5 → 6.25 → 3.125. 높이 ÷ 210,000의 몫 = 반감 횟수
- 블록 검증 4가지: 이전 블록 해시 · 트랜잭션 유효성 · 머클 루트 · 작업 증명

## 5. 이더리움

- 비트코인 한계: RBF + Zero Confirmation 이중지불, 51% 공격, 스크립트 **튜링 불완전**
- 비탈릭 부테린 백서(2013), 개빈 우드 황서(2014), 메인넷 2015.07.30. **계정 모델**

| | EOA | CA |
|---|---|---|
| 통제 | 개인키 | 컨트랙트 코드 |
| 코드 | 없음 | 있음 (codeHash, 변경 불가) |
| 트랜잭션 생성 | 가능 — **모든 트랜잭션의 출발점** | 불가 (받은 호출에 응답만) |
| 주소 | 공개키 Keccak-256의 **마지막 20바이트** (EIP-55 체크섬) | 배포자 주소 + nonce 해시 |

- 계정 필드: **nonce, balance(Wei), codeHash, storageRoot**
- **가스**: 튜링 완전 → 무한 루프(정지 문제) 방지. 수수료 = 사용 가스 × 가스 가격, 남은 가스 환불. **가스 소진 시 상태 reverted + 환불 X**
- **EIP-1559**: 총 = Gas Used × (Base + Priority). **Base Fee는 소각**, Priority Fee는 검증자. Max Fee와의 차이는 반환
- The Merge 이후 **실행 계층(EL, EVM·상태)** + **합의 계층(CL, 제안·투표·최종성)**. 슬롯 12초, 에포크 32슬롯, 32 ETH, **LMD-GHOST**(분기), **Casper FFG**(2/3 최종성)
- 트릴레마: 탈중앙성 · 확장성 · 보안성 / Optimistic 롤업(맞다고 가정, 이의 기간) vs **ZK 롤업**(영지식 증명으로 즉시 검증) / **블롭(EIP-4844)**: L2 데이터용 저렴한 임시 공간, 데이터 가용성만
- 도구: Remix(웹 IDE, Remix VM), MetaMask(지갑, 비밀복구구문 보관), Sepolia(테스트넷), Faucet(테스트 ETH), Etherscan

## 6. 솔리디티 (8주차)

- **정적 타입**, 컴파일 시 타입 검사, **C 계열** 구문. 솔리디티 모노컬처: 상호운용·조합성↑ / 공통 취약점이 생태계 전체에
- 첫 줄 `// SPDX-License-Identifier: MIT` — 없으면 **경고**(오류 X)
- **pragma**: `^0.8.3` = 0.8.3 이상 0.9.0 미만 / `>0.8.3` = 초과 / `0.8.3` = 정확히 그 버전
- `constructor` : **배포 시 한 번만** 실행. `public` 상태 변수는 getter 자동 생성, `private` 은 X. `internal` 함수는 외부(Remix 버튼)에서 호출 불가
- 변수: 상태(컨트랙트 안·함수 밖, 스토리지에 영구, 가스 큼) / 지역(함수 안, 스택·메모리) / 전역( `msg.sender` , `block.timestamp` )
- **msg.sender** = 나를 직접 호출한 주소 / **tx.origin** = 트랜잭션을 시작한 EOA

| EVM 저장 영역 | 특징 | 대입 규칙 (참조 타입) |
|---|---|---|
| 스토리지 | 상태 변수, 영구 | **같은 영역끼리 = 참조** (memory→memory, storage→storage)<br>**다른 영역 사이 = 복사** (memory↔storage, calldata→memory/storage) |
| 메모리 | 함수 실행 중 임시 | |
| calldata | 외부 호출 입력, 읽기 전용 | |
| 스택 | 내부 계산용, 1024단계 이상 예외 | |

- **값 타입**: bool, uint/int, address, enum, bytes1~32 / **참조 타입**: string, 배열, struct, mapping, bytes(동적) — memory 등 위치 명시
- `uint` = uint256, int8 –128~127, uint8 0~255, 지수 `**`
- 정수 나눗셈은 버림: 변수 `3 / 2 * 2` = 2, **리터럴 상수식**은 정확히 계산돼 3. `16 << 4` = 256, `16 >> 3` = 2
- enum 기본값 = 첫 항목(0). `uint(Colors.Green)` = 2
- address: `balance` (wei), `transfer` (실패 시 revert), `send` (실패 시 false), `delegatecall` (**코드만 빌리고 저장소는 내 것**). 송금은 `address payable`
- `this.e.selector` = 함수의 **4바이트** 식별자, `this.e` 는 외부 함수 타입

## 꼭 풀어볼 문제

기존 100제에서 출제 가능성이 높은 ⭐ 문제 중 계산·추적·암기 포인트가 겹치지 않게 골랐습니다. 답은 맨 뒤에 모았습니다. [출처 번호]로 원래 100제에서 찾을 수 있어요.

### ① 탄생 · 기초

1. 순서 배열 다음 사건을 시간 순서대로 배열하시오.
(가) 닉슨 쇼크 (나) 브레튼우즈 협정 (다) 서브프라임 모기지 사태 (라) 뉴딜 정책(금본위제 폐지) [100제(1) 3]

2. 서술 **이중 지불(Double spending)**의 정의를 쓰고, 블록체인에서 이중 지불이 발생할 수 있는 원인을 설명하시오. [100제(1) 17]

3. 순서 배열 블록체인 거래 동작 과정을 순서대로 배열하시오.
(가) 합의를 통해 대표 블록 선정 (나) 트랜잭션 작성 (다) 블록체인에 기록 (라) 네트워크 전파 (마) 각 노드에서 블록 생성 (바) 완성 블록 전파 [100제(1) 19]

### ② 암호기술

4. 매칭 해시 함수의 보안 성질과 정의를 연결하시오.
(A) 역상 저항성 (B) 제2역상 저항성 (C) 충돌 저항성
㉠ h(x)=h(x′), x≠x′인 **쌍 (x, x′)**을 찾기 어려움
㉡ y가 주어졌을 때 h(x)=y인 x를 찾기 어려움
㉢ x가 주어졌을 때 h(x)=h(x′), x≠x′인 x′를 찾기 어려움 [100제(1) 28]

5. 계산 트랜잭션 8개로 머클 트리를 만들었다. 특정 트랜잭션이 포함되어 있음을 증명하는 머클 증명에 필요한 형제(sibling) 해시는 몇 개인가? (루트 해시 제외) [100제(1) 33]

6. 단답 트랜잭션 Tx1~Tx4로 머클 트리를 만들었다(해시1~해시4 → 해시12, 해시34 → 루트해시). **Tx3**의 머클 증명을 구성하는 값을 모두 쓰시오. [100제(2) 23]

### ③ 합의 알고리즘

7. 서술 하드포크와 소프트포크의 차이를 **호환성**과 **체인 유지 여부** 관점에서 설명하고, 각각의 예를 쓰시오. [100제(1) 50]

8. 계산 PBFT에서 악의적인 노드가 최대 f = 2개 존재해도 합의에 도달하려면 최소 몇 개의 노드가 필요한가? 또한 Commit 단계로 넘어가려면 일치하는 메시지를 최소 몇 개 받아야 하는가? [100제(1) 56]

9. 순서 배열 PBFT 동작 단계를 순서대로 배열하시오.
(가) Commit (나) Pre-Prepare (다) Reply (라) Request (마) Prepare [100제(1) 58]

10. 서술 PoW 51% 공격으로 이중 지불이 일어나는 과정을 "고의 분기"와 "Longest Chain 규칙"이라는 단어를 넣어 설명하시오. [100제(1) 79]

11. O/X 최장 체인 규칙(Longest Chain Rule)은 시간이 지날수록 한 가지 체인으로 수렴하므로 **결정적(즉시) 최종성**을 제공한다. [100제(2) 33]

12. 객관식 합의 알고리즘 비교로 **옳지 않은** 것은?
① PoW의 분기 선택은 유효한 체인 중 누적 작업량이 최대인 체인이다
② PoS(이더리움)는 LMD-GHOST로 지분 가중 투표를 반영해 분기를 선택한다
③ PBFT는 참여자가 늘수록 통신량이 줄어 대규모 개방형 네트워크에 적합하다
④ DPoS는 적은 합의 참여자로 통신 부담이 감소하지만 대표 집중·담합 위험이 있다 [100제(2) 41]

### ④ 비트코인

13. 빈칸 비트코인 주소 생성 과정: ECDSA 개인키( ① 비트) → 타원곡선 상수곱셈으로 공개키( ② 비트) → ( ③ ) 해시 → ( ④ ) 해시로 160비트 값 생성 → ( ⑤ ) 인코딩 [100제(1) 60]

14. 계산 비트코인 블록 헤더의 각 필드와 크기(바이트)를 모두 쓰고 합계를 구하시오. [100제(1) 65]

15. 계산 Wallet A가 가진 UTXO는 0.5 BTC와 0.3 BTC 두 개이다. A가 B에게 0.6 BTC를 보내고 수수료로 0.01 BTC를 내려 한다. 이 트랜잭션의 입력, 출력(받는 사람별 금액)을 쓰시오. [100제(1) 69]

16. 계산 nBits = 0x1d00ffff일 때 ① E와 C 값을 구하고 ② 타겟 T = C × 2^(8(E−3))을 32바이트 16진수로 표현했을 때 C 뒤에 붙는 0의 바이트 수를 쓰시오. ③ 채굴 성공 조건을 쓰시오. [100제(1) 75]

17. 계산/단답 ① 비트코인 난이도는 몇 블록마다 재계산되며 이는 약 몇 주인가? ② 블록 보조금은 몇 블록(약 몇 년)마다 절반이 되는가? ③ 평균 블록 생성 시간은? [100제(1) 76]

18. 계산 A의 지갑에는 2 BTC, 1.5 BTC, 0.7 BTC짜리 UTXO가 있다. A가 B에게 3 BTC를 보내고 수수료로 0.05 BTC를 지불하려 한다. 2 BTC와 1.5 BTC UTXO를 입력으로 쓴다면 출력은 어떻게 구성되는가? 거래 후 A의 잔액은? [100제(2) 49]

19. 계산 최초 블록 보조금이 50 BTC일 때, 블록 높이 450,000에서의 블록 보조금은? [100제(2) 51]

20. 계산 nBits = 0x17034219일 때 ① E(10진수)와 C를 구하고 ② 32바이트 타겟 T를 16진수로 썼을 때 C 앞의 0 바이트 수와 C 뒤의 0 바이트 수를 각각 구하시오. [100제(2) 52]

### ⑤ 이더리움

21. 계산 gas limit 50,000, gas price 20 gwei인 트랜잭션이 실제로 21,000 gas를 사용했다. ① 최대 수수료 ② 실제 수수료 ③ 환불되는 금액을 gwei와 ETH로 구하시오. [100제(1) 85]

22. 표 완성 EOA와 CA를 비교하시오.

| 항목 | EOA | CA |
|---|---|---|
| 통제 수단 | ( ① ) | ( ② ) |
| 코드 저장 | ( ③ ) | ( ④ ) |
| 트랜잭션 스스로 생성 | ( ⑤ ) | ( ⑥ ) |

[100제(1) 87]

23. 계산 EIP-1559 트랜잭션이 21,000 gas를 사용했고 Base Fee 30 gwei, Priority Fee 2 gwei이다. ① 총 수수료 ② 소각되는 양 ③ 검증자가 받는 양을 gwei로 구하시오. [100제(1) 94]

24. 빈칸 이더리움 PoS에서 ( ① ) ETH 이상 예치하면 검증위원 후보가 된다. 매 슬롯( ② 초)마다 제안자가 블록을 제안하고, ( ③ )개 슬롯으로 이루어진 에포크마다 투표한다. 분기 시 ( ④ ) 프로토콜로 메인 체인을 정하고, ( ⑤ ) 프로토콜로 스테이킹 지분 2/3 투표를 통해 최종성을 보장한다. [100제(1) 95]

25. 서술 블록체인 트릴레마를 설명하고, Optimistic Rollup과 ZK Rollup의 차이 및 블롭(EIP-4844)의 역할을 쓰시오. [100제(1) 96]

26. 계산 gas limit 100,000, gas price 15 gwei인 트랜잭션이 60,000 gas를 사용하고 정상 종료되었다. ① 실제 수수료(gwei, ETH) ② 환불되는 금액(gwei)을 구하시오. 같은 트랜잭션이 무한 루프로 실패했다면 수수료는? [100제(2) 63]

27. 계산 EIP-1559 트랜잭션: Max Fee 30 gwei, Base Fee 20 gwei, Priority Fee 3 gwei, 사용 가스 50,000. ① 총 수수료 ② 소각량 ③ 검증자 수취액 ④ Max Fee 기준 대비 돌려받는 금액을 gwei로 구하시오. [100제(2) 72]

### ⑥ 솔리디티 (8주차)

28. O/X 솔리디티는 동적 타입 언어여서 변수의 타입을 실행 시점에 검사하며, 구문은 Python에 가깝다. [100제(2) 84]

29. 판별 각 pragma 지시문에 대해 컴파일러 버전 0.8.2, 0.8.3, 0.8.25, 0.9.0 중 **호환되는 것을 모두** 고르시오.
(a) `pragma solidity ^0.8.3;`
(b) `pragma solidity >0.8.3;`
(c) `pragma solidity 0.8.3;`
(d) `pragma solidity 0.8.2 || 0.8.3;` [100제(2) 87]

30. 코드 해석 다음 코드에 대해 답하시오.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.6.8;
import "./MyLib.sol";

contract HelloWorld {
    string private greet;

    constructor(){
        greet = "Hello World";
    }

    function getString() public view returns(string memory){
        return greet;
    }
}
```

① 이 코드가 정상 동작하는 컴파일러 버전은? ② 첫 줄을 생략하면 어떻게 되는가? ③ `constructor`는 언제, 몇 번 실행되는가? ④ `greet`를 외부에서 읽으려면 왜 `getString()`이 필요한가? [100제(2) 88]

31. 매칭 EVM의 데이터 저장 영역과 설명을 연결하시오.
영역: (A) 스토리지 (B) 메모리 (C) 호출데이터(calldata) (D) 스택
설명: ㉠ 함수 실행 중에만 존재하는 휘발성 임시 공간 ㉡ 컨트랙트 상태 변수가 영구히 저장되는 공간 ㉢ EVM 내부 계산용 임시 공간, 1024단계 이상이면 예외 ㉣ 외부에서 함수를 호출할 때 입력값이 저장되는 읽기 전용 임시 공간 [100제(2) 89]

32. 시나리오 사용자 Alice(EOA)가 컨트랙트 A의 함수를 호출했고, 그 함수 안에서 컨트랙트 B의 함수를 호출했다. **컨트랙트 B 안에서** `msg.sender`와 `tx.origin`의 값은 각각 누구의 주소인가? [100제(2) 90]

33. 분류 다음 자료형을 값 타입과 참조 타입으로 나누시오.
`bool`, `uint`, `address`, `string`, 배열, `struct`, `mapping`, `enum`, `bytes32`, `bytes` [100제(2) 93]

**34.** 표 완성 참조 타입 데이터를 대입할 때 복사되는지(사본 생성), 참조되는지(같은 공간 공유) 쓰시오.

| 소스 → 타겟 | 결과 |
|---|---|
| memory → memory | ( ① ) |
| memory → storage | ( ② ) |
| storage → memory | ( ③ ) |
| storage → storage | ( ④ ) |
| calldata → memory | ( ⑤ ) |

[100제(2) 94]

**35.** 코드 해석 Remix에서 각 함수를 호출했을 때의 반환값을 쓰시오.

```
contract Integers{
    function getTwo() public pure returns(uint){
        uint a = 3;
        uint b = 2;
        return a / b * 2;
    }

    function getThree() public pure returns(uint){
        return 3 / 2 * 2;
    }

    function shift() public pure returns(uint[2] memory){
        uint[2] memory a;
        a[0] = 16 << 4;
        a[1] = 16 >> 3;
        return a;
    }
}
```

[100제(2) 96]

**36.** 객관식 주소 타입의 멤버에 대한 설명으로 **옳지 않은** 것은?

① `balance` 는 계정 주소에 있는 이더를 wei 단위로 반환한다
② `transfer` 는 송금에 실패하면 트랜잭션이 되돌려진다
③ `send` 는 송금에 실패해도 트랜잭션을 되돌리지 않고 `false` 를 반환한다
④ `delegatecall` 은 다른 컨트랙트의 코드와 저장소를 모두 그 컨트랙트 것으로 사용한다
⑤ `transfer` , `send` , 송금 목적의 `call` 은 `address payable` 타입에서만 사용할 수 있다 [100제(2) 97]

**37.** 코드 해석 배포 직후 ① `getColor()` ② `getDefaultcolor()` 를 호출한 결과와, ③ `setColor()` 를 호출한 뒤 `getColor()` 의 결과를 쓰시오.

```
contract Enum {
    enum Colors {Red, Blue, Green}

    Colors color;
    Colors constant defaultColor = Colors.Green;

    function setColor() public {
        color = Colors.Blue;
    }

    function getColor() public view returns (Colors){
        return color;
    }

    function getDefaultcolor() public pure returns(uint){
        return uint(defaultColor);
    }
}
```

[100제(2) 99]

**38.** 코드 해석 다음 코드에 대해 답하시오.

```
contract Selector{
    function e() external pure returns(bytes4){
        return this.e.selector;
    }

    function f() public pure returns(uint){
        return g();
    }

    function g() internal pure returns(uint){
        return 100;
    }
}
```

① `f()` 의 반환값은? ② `this.e` 와 `g()` 는 각각 내부/외부 함수 타입 중 무엇인가? ③ `e()` 가 반환하는 `selector` 는 무엇이며 몇 바이트인가? ④ 외부 계정이 Remix에서 `g()` 를 직접 호출할 수 있는가? [100제(2) 100]

## 정답 및 해설

1. **(라) → (나) → (가) → (다)**: 뉴딜(1933) → 브레튼우즈(1944) → 닉슨 쇼크(1971) → 서브프라임(2008)

2. 동일한 화폐를 두 번 사용하는 행위입니다. 디지털 데이터는 복사·재사용이 가능하고, 데이터 전파 과정의 **손실·지연으로 노드 간 장부가 불일치**하면 최신 블록을 받지 못한 노드가 이중 지불을 발견하지 못하고 거래를 승인할 수 있습니다. → **합의**로 대표 장부를 선정·동기화해 해결합니다.

3. **(나) → (라) → (마) → (바) → (가) → (다)**

4. **(A)-㉡, (B)-㉢, (C)-㉠** — 지문 비유: 지문만 보고 누구인지(역상) / Alice와 같은 지문의 사람(제2역상) / 지문이 같은 아무 두 사람(충돌)

5. **3개** — $\log_2 8 = 3$ (각 높이마다 형제 해시 1개)

6. **{해시4, 해시12, 루트해시}** — 해시3과 해시4로 해시34를 만들고, 해시12와 합쳐 루트를 계산한 뒤 루트해시와 비교합니다.

7. **하드포크**: 규칙이 바뀌어 옛 노드와 **호환되지 않음**. 업그레이드 불일치가 계속되면 **두 체인으로 분리**되어 공존합니다(비트코인/비트코인 캐시, 이더리움/이더리움 클래식). **소프트포크**: 규칙이 더 엄격해져 옛 노드도 부분적으로 수용(**하위 호환**)하고 **하나의 체인으로 유지**됩니다(SegWit).

8. **3f+1 = 7개** 노드, **2f+1 = 5개** 메시지

9. **(라) → (나) → (마) → (가) → (다)**: Request → Pre-Prepare → Prepare → Commit → Reply

10. 공격자는 UTXO#1로 상점 B와 C에게 보내는 두 트랜잭션을 만듭니다. B 거래를 먼저 전파해 블록에 포함시키고 상품을 받습니다. 동시에 C 거래가 포함된 블록을 비공개로 만들어 **고의 분기**를 일으키고, 과반 해시파워로 실제 체인보다 긴 체인을 만든 뒤 공개합니다. **Longest Chain 규칙**에 따라 공격자의 체인이 채택되고 B 거래는 취소됩니다.

11. **X** — 최장 체인 규칙은 **확률적 최종성**을 제공합니다. 즉시·결정적 최종성은 PBFT(2/3 이상 투표)의 특징입니다.

12. **③** — PBFT는 참여자가 늘면 메시지가 폭증해 성능이 떨어지므로 **허가형** 환경에 적합합니다.

13. ① 256 ② 512 ③ SHA-256 ④ RIPEMD-160 ⑤ Base58 (메인넷 접두어 0x00)

14. 버전(4) + 이전 블록해시(32) + 머클루트(32) + 타임스탬프(4) + 난이도(4) + 넌스(4) = **80바이트** (트랜잭션 카운터는 블록 해시에 포함되지 않음)

15. 입력: UTXO 0.5 + 0.3 = **0.8 BTC** (두 UTXO 모두 소비) / 출력: **B에게 0.6 BTC**, **A에게 거스름돈 0.19 BTC** / 수수료 = 0.8 − 0.79 = 0.01 BTC

16. ① E = 0x1d = **29**, C = **0x00ffff** ② E−3 = **26바이트** ③ 블록 해시 **h < T**
(참고: T = 0x00000000ffff0000…0000 — 앞쪽 32−29 = 3바이트가 0)

17. **① 2016블록, 약 2주 ② 210,000블록, 약 4년 ③ 약 10분**

18. 입력 3.5 BTC → 출력: **B에게 3 BTC**, **A에게(거스름) 0.45 BTC**. 차액 0.05 BTC가 수수료입니다. 사용한 두 UTXO는 파기되고, A의 잔액은 남은 UTXO의 합 0.7 + 0.45 = **1.15 BTC**입니다.

19. 450,000 ÷ 210,000 = 2.14… → 반감기 **2회** 경과 → 50 → 25 → **12.5 BTC**

20. ① E = 0x17 = **23**, C = **0x034219** ② C 뒤의 0: E − 3 = **20바이트**, C 앞의 0: 32 − 3 − 20 = **9바이트**
T = 0x0000000000000000000**034219**0000…(20바이트)

21. ① 50,000 × 20 = **1,000,000 gwei = 0.001 ETH** ② 21,000 × 20 = **420,000 gwei = 0.00042 ETH** ③ **580,000 gwei = 0.00058 ETH** (1 gwei = $10^{-9}$ ETH)

22. ① 개인키 ② 컨트랙트 코드 ③ 불가 ④ 가능 ⑤ 가능 ⑥ 불가(받은 트랜잭션에 응답만 함)

23. ① 21,000 × (30 + 2) = **672,000 gwei** ② 21,000 × 30 = **630,000 gwei** (Base Fee는 소각) ③ 21,000 × 2 = **42,000 gwei** (Priority Fee)

24. **① 32 ② 12 ③ 32** (약 6.4분) **④ LMD-GHOST ⑤ Casper FFG**

25. **트릴레마**: 탈중앙성·확장성·보안성을 동시에 완벽히 달성하기 어렵다는 것입니다. **Optimistic Rollup**: 올바르다고 가정하고 L1에 전달하며 이의 제기가 없으면 확정합니다. **ZK Rollup**: 영지식 증명으로 오류 없음을 입증해 L1에서 빠르게 검증합니다. **블롭**: 블록에 저렴하고 일시적인 대용량 공간을 추가해 L2 롤업 데이터를 저장하며, 실행에는 관여하지 않고 **데이터 가용성**만 보장합니다.

26. ① 60,000 × 15 = **900,000 gwei = 0.0009 ETH** ② (100,000 – 60,000) × 15 = **600,000 gwei** 환불. 무한 루프라면 가스를 모두 소진하므로 100,000 × 15 = **1,500,000 gwei(0.0015 ETH)** 를 내고 환불은 없으며, 상태 변경은 취소(reverted)됩니다.

27. 실제 가스 단가 = 20 + 3 = 23 gwei
① 50,000 × 23 = **1,150,000 gwei** (0.00115 ETH)
② 50,000 × 20 = **1,000,000 gwei** 소각
③ 50,000 × 3 = **150,000 gwei**
④ (30 – 23) × 50,000 = **350,000 gwei**

28. **X** — 솔리디티는 **정적 타입** 언어로 **컴파일 시** 타입을 검사하며, 구문은 **C에 가깝습니다**. 상속·인터페이스·라이브러리 같은 객체지향 요소를 포함합니다.

29. (a) **0.8.3, 0.8.25** — ^0.8.3 = 0.8.3 이상 0.9.0 미만
(b) **0.8.25, 0.9.0** — 0.8.3 초과
(c) **0.8.3** — `=0.8.3` 과 같은 의미
(d) **0.8.2, 0.8.3**

30. ① **정확히 0.6.8만** ② 솔리디티 0.6 이상에서는 SPDX 라이선스 식별자가 없으면 **경고**가 납니다(오류는 아님). ③ 컨트랙트가 **배포될 때 한 번만** 최초 실행되는 초기화 함수입니다. ④ `greet` 가 **private**이라 getter가 자동 생성되지 않기 때문입니다. `public` 이었다면 `greet()` getter가 자동으로 만들어집니다.

31. **A-㉡, B-㉠, C-㉣, D-㉢**

32. `msg.sender` = **컨트랙트 A의 주소**(B를 직접 호출한 주소), `tx.origin` = **Alice의 주소**(트랜잭션을 시작한 원래 주소)

33. 값 타입: **bool, uint, address, enum, bytes32**(고정 크기 바이트 배열) / 참조 타입: **string, 배열, struct, mapping, bytes**(동적 크기 바이트 배열)

34. ① **참조** ② **복사** ③ **복사** ④ **참조** ⑤ **복사** — 같은 영역끼리는 참조(얕은 복사), 다른 영역 사이는 복사입니다. calldata → storage도 복사입니다.

35. getTwo → **2**: 변수 a/b는 uint이므로 3/2 = 1(소수점 버림), 1 × 2 = 2
getThree → **3**: 리터럴끼리의 상수 식은 컴파일 시 정확한 값으로 계산되어 3/2 × 2 = 3
shift → **[256, 2]**: $16 \times 2^4 = 256$, $16 \div 2^3 = 2$

36. ④ — `delegatecall` 은 다른 컨트랙트의 **코드만 빌려 쓰고, 저장소는 현재 컨트랙트 것**을 사용합니다. 참고로 `transfer` / `send` 는 0.8.31부터 deprecated입니다.

37. ① **0** (Red) — setColor()를 호출하지 않으면 기본값 0 ② **2** (Green) ③ **1** (Blue)

38. ① **100** ② `this.e` 는 주소(현재 컨트랙트 `this` )를 통해 호출하는 **외부 함수 타입**, `g()` 는 현재 함수 안에서 참조하는 **내부 함수 타입** ③ 함수의 **4바이트 식별자**( `bytes4` ) ④ **불가능** — `internal` 함수는 외부에서 호출할 수 없으므로 Remix에 버튼도 생성되지 않습니다.